

Charting ORCA

The full problem brief, system architecture, and a five-week build plan for Team ASTID's Marine EcOsystem Reasoning with Collaborative Agents entry.

Prepared for — Team ASTID (6 members)

Problem statement — SIH26176, Indian Space Research Organisation (ISRO)

Compiled — 27 August 2026

Idea submission deadline — 20 September 2026

06

TEAM MEMBERS

05

BUILD WEEKS

08

ROADMAP PHASES

The Problem Statement

PS ID	CATEGORY	DEPARTMENT	DEADLINE	IDEAS SUBMITTED
SIH26176	Software	Dept. of Space / ISRO	20 Sep 2026	0 / 500

TITLE

ORCA — Marine EcOsystem Reasoning with Collaborative Agents.

BACKGROUND

The marine ecosystem underpins livelihoods, food security, biodiversity, maritime transport, coastal resilience, and the blue economy. Every day, ISRO and other global agencies generate vast volumes of satellite Earth Observation and oceanographic data — sea surface temperature, chlorophyll concentration, weather forecasts. Fishermen, researchers, coastal authorities, disaster-management agencies, and maritime operators all need timely access to this information for operational decisions, but the growing volume and complexity of marine data calls for an intelligent conversational platform: one where users can ask questions, explore scenarios, and get synthesized, evidence-based recommendations in context. Advances in agentic AI, conversational intelligence, and geospatial technology make it possible to answer questions like "Where is the nearest Potential Fishing Zone today?" or "Is it safe to venture into the sea tomorrow morning?" with explainable, context-aware reasoning.

OBJECTIVE

Develop an agentic AI-powered conversational platform that lets users access, analyse, and reason over marine information in natural language. The platform should autonomously interpret intent, decompose complex requests into executable tasks, coordinate multiple specialised AI agents, retrieve relevant marine and geospatial datasets, perform spatial-temporal reasoning, and synthesize actionable recommendations — integrating satellite Earth Observation products, GIS layers, weather services, oceanographic observations, and public marine advisories.

TYPICAL USER QUERIES

- "Where is the nearest Potential Fishing Zone (PFZ) today?"
- "Is it safe to venture into the sea tomorrow morning?"
- "What are the tide, weather, and sea conditions near my fishing location?"
- "Are there any lightning or cyclone alerts in my area?"
- "Which regions show high chlorophyll concentration and favourable sea surface temperature?"

- *"What is the safest route for a fishing vessel given current weather and sea state?"*
- *"Why has fish productivity declined in a particular coastal region?"*
- *"Which fishing zones should be avoided due to hazardous conditions or geofencing restrictions?"*

EXPECTED CAPABILITIES

- Understand user intent expressed in natural language, and support contextual, multi-turn conversation for refining queries.
- Automatically detect the query's language and respond in kind, with emphasis on Indian regional languages.
- Autonomously discover, retrieve, and integrate satellite, marine, meteorological, and geospatial datasets.
- Perform spatial, temporal, and contextual reasoning across heterogeneous data sources.
- Generate explainable, evidence-based recommendations with maps, charts, and geospatial visualizations.
- Proactively alert fishermen to adverse weather, high waves, lightning, cyclones, and other hazards.
- Provide geofencing notifications near international maritime boundaries, restricted waters, or marine protected areas.
- Assist with route optimization, safe navigation, and operational planning against forecast conditions.
- Deliver every recommendation with the supporting evidence and reasoning behind it.

SUGGESTED ARCHITECTURE (OFFICIAL BRIEF)

A modular multi-agent design is encouraged, with specialised agents for planning, marine-data discovery, weather intelligence, ocean analytics, geospatial reasoning, risk assessment, visualization, reporting, and user interaction — demonstrating autonomous collaboration among agents on complex marine-intelligence problems, wrapped in an intuitive conversational experience.

DATASET — none prescribed by ISRO; the team sources public-domain satellite Earth Observation, GIS, weather, and marine-advisory data itself (see the Data & Tool Layer in the architecture section later in this dossier).

How This Dossier Is Built

Everything from here on — the roadmap, the diagrams, and the five-week plan — is original planning material prepared for Team ASTID. It is not part of the official ISRO listing. A few working assumptions were made to turn the brief into a concrete schedule; flag any of these to Dhrubojyoti if they don't match reality and the plan gets adjusted accordingly.

Assumptions Baked Into This Plan

- The five build weeks start this week (27 August 2026) and run through 30 September 2026 — Week 4 lands on the 20 September idea-submission deadline.
- Tech stack: **LangGraph** for multi-agent orchestration, **FastAPI** backend, **Supabase** for persistence, **Next.js** + **Leaflet/Mapbox** frontend — carried over from ASTID's existing stack.
- Primary data sources: **MOSDAC** (ISRO's own ocean/weather portal) for SST, chlorophyll, and ocean current; **INCOIS ERDDAP/WebGIS** for Potential Fishing Zone advisories and hazard alerts; a public GIS layer for maritime boundaries and marine protected areas.
- The team is starting this build fresh — no existing ORCA code assumed going into Week 1.
- "Committed to GitHub" below means a real, reviewable commit on the team repo by end of that week — not just local, uncommitted work.

Eight Phases, In Order

This is the definition-of-done checklist for the whole build — phase by phase, regardless of calendar week. The five-week plan later in this dossier maps each week to the phases it covers, but this list is what "finished" actually means for each part of the system.

00

Research & Discovery

Goal: nobody discovers a blocking unknown after Week 1.

- ☐ Read the full official problem statement and extract every mandatory requirement into a single checklist
- ☐ Get real, working access to MOSDAC's Data Download API and INCOIS's ERDDAP endpoint — not just reading documentation
- ☐ Study 2–3 reference projects or papers on agentic geospatial systems for architectural patterns
- ☐ Decide which of the eight sample queries in the brief the MVP must answer end-to-end
- ☐ Lock the tech stack — no swapping frameworks after Week 1

01

Architecture & Design

Goal: everyone can build their piece without waiting on someone else's undecided interface.

- ☐ Design the multi-agent graph — planner, specialists, synthesizer — and how control passes between them
- ☐ Define the shared state schema every agent reads and writes
- ☐ Design the Supabase table schema (sessions, messages, geofence zones)
- ☐ Design the API contract — request/response shape for every endpoint
- ☐ Sketch the core UI screens — chat window, map view, alerts panel
- ☐ Write it all down in an ARCHITECTURE.md the whole team has read

02

Data Layer

Goal: every agent has real data to call — no mocked responses left by Week 3.

- ☐ Build and unit-test a callable function for every external data source
- ☐ Add caching so repeated queries don't hammer external APIs
- ☐ Add a fallback/cached dataset for when a live API is down or slow
- ☐ Build the geofencing dataset — maritime boundaries, marine protected areas, restricted zones
- ☐ Compile an evaluation set: realistic queries with the answer they should produce

03**Agent Development**

Goal: each specialist agent gives a correct, explainable answer in isolation before they're wired together.

- ☐ Build the Planner Agent — query decomposition and task routing
- ☐ Build the Weather Agent, Ocean Agent, and Risk/Geofence Agent
- ☐ Build the Synthesizer Agent — merges specialist outputs into one coherent, evidence-cited answer
- ☐ Add language detection and multilingual response generation
- ☐ Every answer must state which data it used, not just assert a conclusion

04**Backend & Integration**

Goal: the agent graph is reachable from the outside world, reliably.

- ☐ Build the chat/session API endpoint
- ☐ Wire the LangGraph agent pipeline into that endpoint, replacing any mocked responses
- ☐ Persist every conversation turn to Supabase
- ☐ Add error handling — a failed external API call should degrade gracefully, never crash the chat
- ☐ Deploy the backend to a staging environment

05**Frontend**

Goal: a stranger can use this without anyone explaining it to them first.

- ☐ Build the chat interface — message history, loading states, source/evidence display
- ☐ Build the map view — renders zones, alerts, and recommended routes from live backend data
- ☐ Build the alerts/notifications panel
- ☐ Add a language indicator or selector to the UI
- ☐ Mobile-responsive pass
- ☐ Deploy the frontend

06**Testing & QA**

Goal: no surprises during the live demo.

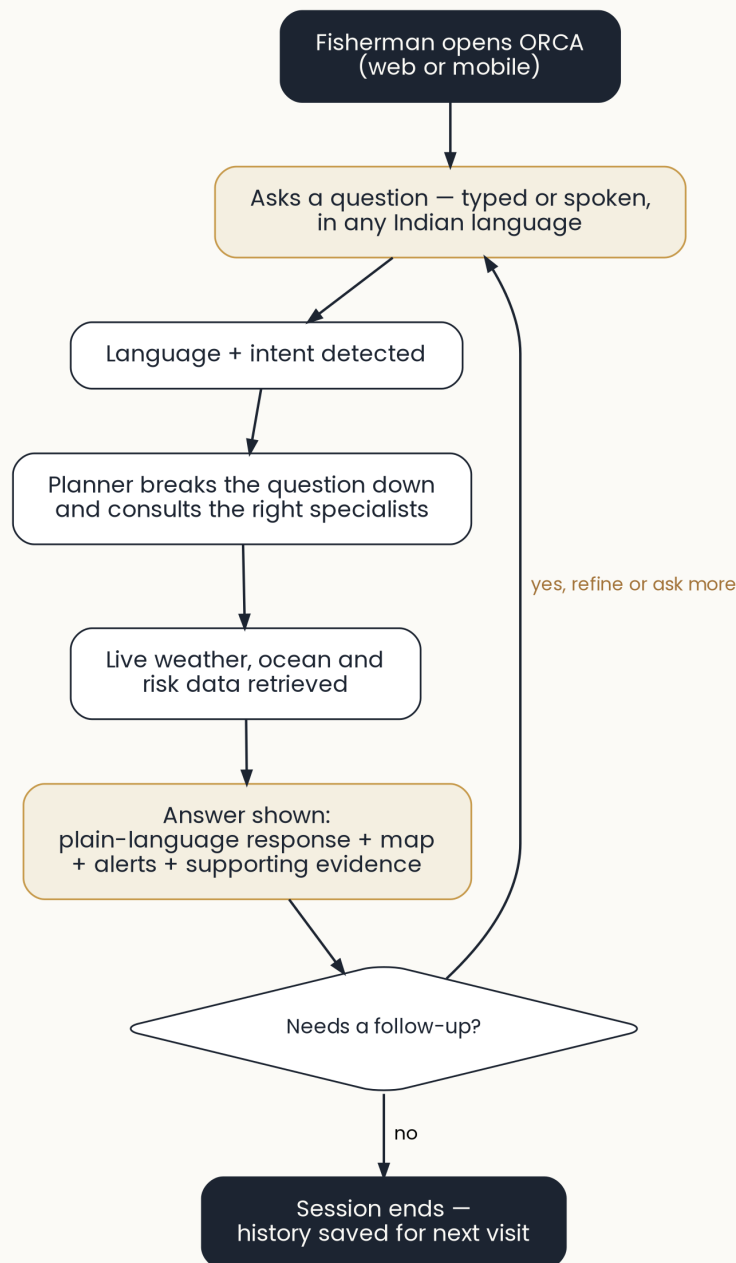
- ☐ Run every one of the brief's eight sample queries end-to-end and confirm a sensible answer
- ☐ Run the full evaluation set and tune prompts based on failures
- ☐ Stress-test behaviour when an external API is slow, down, or returns nothing
- ☐ Check the experience on at least one phone and one desktop browser

Goal: the team walks in rehearsed, not rehearsing.

- ☐ Submit the idea write-up and PPT on the SIH portal before 20 September
- ☐ Finalise the pitch deck and demo script
- ☐ Record a backup demo video in case live internet fails on the day
- ☐ Run at least two full team rehearsals, timed
- ☐ Tag a v1.0 release on GitHub with a complete README

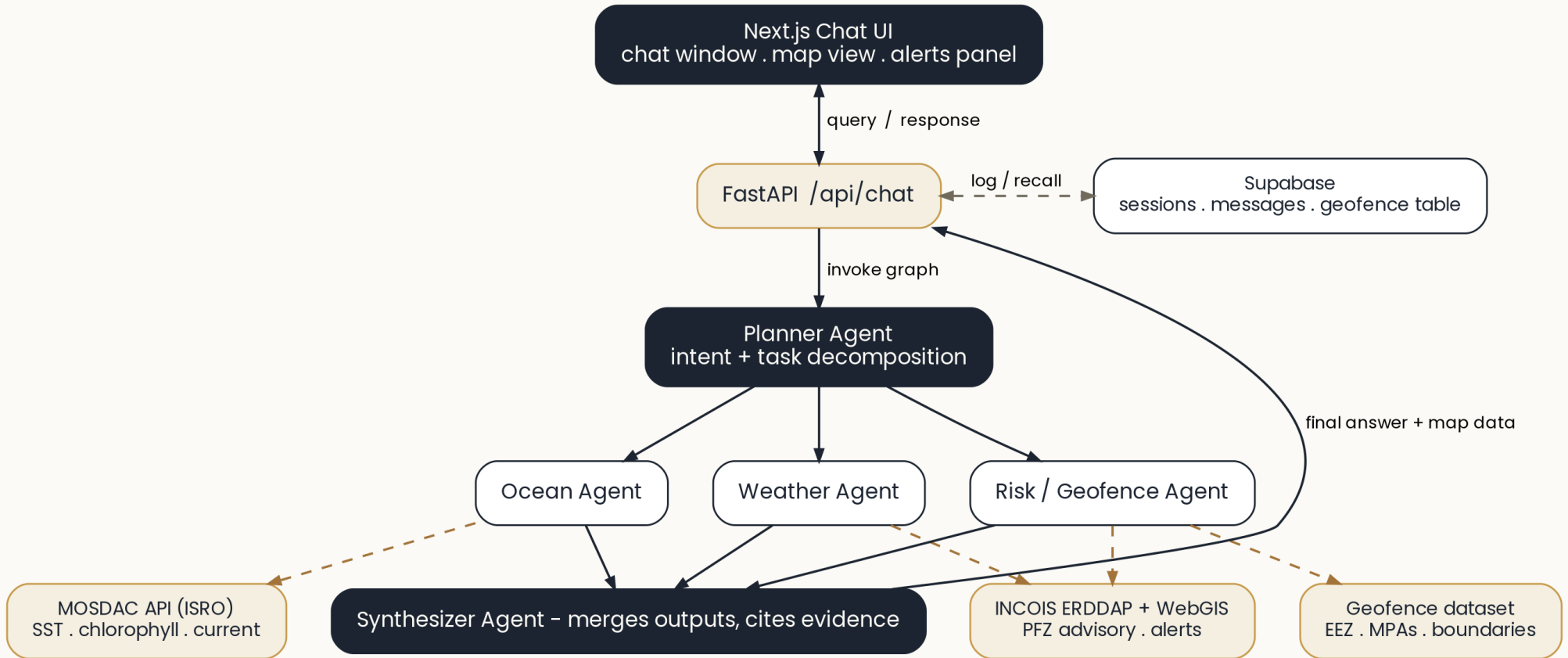
User Flow

What it actually feels like to use ORCA, from the moment someone opens the app to the moment they get an answer they can act on.



A single question can loop through the same path multiple times in one sitting — the system holds context, so “what about tomorrow instead?” works without the user repeating themselves.

Architecture & Data Flow



Solid arrows are the request/response path. Dashed arrows are read-only calls out to external data. The Planner never touches raw data itself — it only ever talks to the three specialist agents, which is what keeps each agent swappable and testable on its own.

What Each Layer Is Responsible For

CLIENT LAYER

The Next.js app is the only thing the user ever sees — a chat window, a map that updates with whatever zones or alerts are relevant to the current answer, and an alerts panel for anything urgent (a cyclone warning, a restricted-zone notice). It never talks to MOSDAC or INCOIS directly; it only ever talks to the FastAPI backend.

API LAYER

A single FastAPI endpoint, `/api/chat`, is the front door. It takes the user's message and session ID, hands the message to the agent graph, and returns whatever the Synthesizer Agent produces. It also reads and writes conversation history to Supabase, so a returning user's context isn't lost.

AGENT ORCHESTRATION LAYER

This is ORCA's core. The Planner Agent reads the query and decides which of the three specialists are actually relevant — a question about fish productivity might only need the Ocean Agent, while "is it safe tomorrow" needs all three. Each specialist agent's only job is to call its own data tool, reason over the result, and hand back a structured finding. The Synthesizer Agent is the only one that talks to the user directly — it combines whatever the specialists returned into one plain-language answer, in the user's own language, and lists what evidence backs it up.

DATA & TOOL LAYER

Every specialist agent calls exactly one kind of external tool: the Weather Agent calls INCOIS's alert and advisory data, the Ocean Agent calls MOSDAC's satellite-derived sea surface temperature and chlorophyll data, and the Risk/Geofence Agent checks both the geofence dataset and INCOIS's Potential Fishing Zone data. None of these tools know about each other — that separation is what lets Tiyyasha and Samprikta build and test their halves independently.

PERSISTENCE

Supabase quietly sits underneath all of it, logging every message and letting the app recall a session's history — it's not part of the reasoning path, just the memory of it.

Five Weeks, Six People

Each week has one goal, and each person has specific, committable work against it. "Commit" below means a real pull request or push to the shared GitHub repo by the end of that week — reviewed by at least Dhrubojyoti before the week closes. Where two people's work depends on each other, that's called out as an **integration checkpoint** at the end of the week — that's the moment to actually sit together, not just message on chat.

WEEK 1	WEEK 2	WEEK 3	WEEK 4	WEEK 5
27 Aug – 2 Sep	3 – 9 Sep	10 – 16 Sep	17 – 23 Sep	24 – 30 Sep

Week 1

27 AUG – 2 SEP · PHASES 00 & 01

Goal: the repo exists, real data access is proven end to end, and the architecture is written down — nobody starts Week 2 guessing at an interface someone else hasn't decided yet.

Dhrubojyoti

LEAD · ARCHITECTURE

- Finalise the LangGraph agent design end to end: planner → three specialists → synthesizer, including the shared state schema passed between them
- Create the GitHub repo, branch strategy, and a project board with one issue per checklist item from the roadmap
- Write ARCHITECTURE.md covering the diagrams and layer responsibilities in this dossier
- Run the Week 1 integration check-in

COMMIT — repo skeleton, ARCHITECTURE.md, populated issue board

Isheeka

BACKEND · SUPABASE

- Create the Supabase project and schema: `users`, `chat_sessions`, `messages`, `geofence_zones`
- Scaffold the FastAPI app with a working health-check endpoint and environment/config setup
- Confirm FastAPI can read and write to Supabase

COMMIT — backend/ FastAPI skeleton + Supabase migration files

Biswanath

FRONTEND

- Scaffold the Next.js app and basic chat UI shell — message list and input box, no live data yet
- Add routing skeleton for the app's main screens
- Add an empty Leaflet/Mapbox map component stub, just rendering a base map

COMMIT — frontend/ Next.js skeleton + chat UI shell + empty map component

Adhiraj

PRESENTATION

- Research the SIH software-track judging criteria and study 2–3 past winning marine or agri decision-support projects for pitch patterns
- Draft version one of the problem–solution–impact narrative for ORCA
- Compile real fishermen's safety pain points from secondary research, to ground the pitch in something true rather than generic

COMMIT — docs/pitch-narrative-v1.md, docs/judging-criteria-notes.md

Tiyasha

DATA & INTEGRATION

- Get MOSDAC's Data Download API working locally and successfully pull one real SST or chlorophyll dataset
- Get INCOIS's ERDDAP endpoint returning real Potential Fishing Zone data for at least one coastal sector
- Document the exact fields and formats each source actually returns — this becomes the contract Samprikta's agents build against

COMMIT — data/mosdac_test_pull.py, data/incois_erddap_test.py, docs/data-sources.md

Samprikta

AGENT DEVELOPMENT

- Set up the LangGraph project skeleton and define the agent state schema alongside Dhrubojyoti
- Build a "hello world" two-agent chain — planner calling one dummy specialist — to prove the orchestration pattern works before real data is involved

COMMIT — agents/ LangGraph skeleton + working dummy two-agent chain + first prompt templates

INTEGRATION CHECKPOINT

Confirm Tiyasha's documented data formats match what Samprikta's agent schema expects, and that Isheeka's Supabase schema matches what Biswanath's frontend will need to render. Fix mismatches now — not in Week 3.

Week 2

3 – 9 SEP · PHASE 02

Goal: the Weather Agent and Ocean Agent work end to end against real data and return structured results — not natural language yet, just correct structured output.

Dhrubojyoti

LEAD · ARCHITECTURE

- Build the Planner Agent's real decomposition logic — given a question, decide which specialists it actually needs
- Start the Synthesizer Agent as a stub that just concatenates specialist outputs, to be replaced with real synthesis in Week 3
- Review every PR merged this week

COMMIT — `agents/planner_agent.py`, `agents/synthesizer_agent.py` (stub)

Isheeka

BACKEND · SUPABASE

- Build the chat session API — create and retrieve conversation history
- Build `/api/chat` returning a mocked agent response for now, with every message persisted to Supabase

COMMIT — `backend/routes/chat.py`, working mocked `/api/chat`, message logging to Supabase

Biswanath

FRONTEND

- Build a working chat interface that sends a message and displays the response and conversation history from the backend
- Get the map rendering real markers from a static sample GeoJSON file

COMMIT — chat UI hitting `/api/chat`, map rendering sample PFZ zone markers

Adhiraj

PRESENTATION

- Talk to or research 3–5 people on real concerns and vocabulary around going out to sea, and fold that into the pitch narrative
- Start drafting the demo video storyboard

COMMIT — `docs/user-research-notes.md`, `docs/demo-script-v1.md`

Tiyasha

DATA & INTEGRATION

- Wrap the MOSDAC and INCOIS pulls into clean, typed functions — e.g. `get_sst(lat, lon, date)`, `get_pfz_zones(sector)`
- Add basic caching so repeated calls don't re-hit external APIs every time
- Write unit tests for each data function

COMMIT — `data/tools/mosdac_tools.py`, `data/tools/incois_tools.py`, tests

Samprikta

AGENT DEVELOPMENT

- Build the Weather Agent — calls Tiya's alert data tools, reasons over hazard conditions, returns structured findings
- Build the Ocean Agent — calls Tiya's SST/chlorophyll tools, identifies favourable zones

COMMIT — `agents/weather_agent.py`, `agents/ocean_agent.py`, tested against Tiya's real functions

INTEGRATION CHECKPOINT

Run one hardcoded query fully through Planner → Weather/Ocean agents → real data → a logged Supabase entry, visible as raw JSON in the frontend. If this loop doesn't close cleanly this week, Week 3 slips.

Week 3

10 – 16 SEP · PHASE 03

Goal: at least three of the brief's eight sample queries produce a correct, natural-language answer end to end, in English.

Dhrubojoyoti

LEAD · ARCHITECTURE

- Finish the real Synthesizer Agent — combine specialist outputs into one coherent answer that cites which data it used
- Wire the complete Planner → Specialists → Synthesizer graph together
- Start testing against the brief's sample queries directly

COMMIT — complete `agents/synthesizer_agent.py`, `agents/graph.py`, passing tests on 3+ sample queries

Isheeka

BACKEND · SUPABASE

- Replace the mocked `/api/chat` response with a real call into the full agent pipeline
- Add the `geofence_zones` table with seed data and a serving endpoint

COMMIT — real agent-pipeline integration in backend, geofence endpoint + seed data

Biswanath

FRONTEND

- Polish the chat UI — loading states, markdown rendering, a visible source/evidence section under each answer
- Connect the map to live backend data instead of the static sample file
- Add a language indicator to the UI

COMMIT — polished chat UI, live map integration, language indicator

Adhiraj

PRESENTATION

- Finalise the demo script and a first version of the slide deck
- Run a full team dry-run of the pitch against whatever's working so far and note the gaps

COMMIT — `docs/slides-v1`, `docs/demo-script-v2.md`

Tiyasha

DATA & INTEGRATION

- Add geofencing boundary data — marine protected areas, the international maritime boundary line, restricted zones — from public GIS sources
- Build the geofence data tool functions
- Start compiling the evaluation query set with expected answers, for use from Week 4 onward

COMMIT — `data/tools/geofence_tools.py`, `data/geofence_boundaries.geojson`, `docs/eval-queries.md`

Samprikta

AGENT DEVELOPMENT

- Build the Risk/Geofence Agent using Tiya's geofence tools plus weather and ocean context
- Add language detection and get the synthesizer responding in the detected language — test at minimum Hindi and Bengali alongside English

COMMIT — `agents/risk_agent.py`, multilingual handling in the synthesizer, tests in 2+ languages

INTEGRATION CHECKPOINT

Run all eight sample queries from the official brief through the full stack — frontend to backend to agent graph to response. Whatever breaks, fix together before the week closes.

Week 4

17 – 23 SEP · PHASES 04 & 07

Goal: submit a strong idea entry by 20 September using the real working system as proof of feasibility, while hardening the build in parallel.

Dhrubojyoti

LEAD · ARCHITECTURE

- Lead the SIH idea write-up, using the working prototype as evidence of feasibility rather than a purely theoretical pitch
- Finalise the architecture diagram for the submission
- Run a full QA pass across the agent pipeline and triage any integration bugs the team finds

COMMIT — bug-fix commits across `agents/`, submission-ready architecture diagram

Isheeka

BACKEND · SUPABASE

- Add error handling and graceful fallbacks for when an external API times out or returns nothing
- Do a performance pass on backend response times
- Deploy the backend to a staging environment

COMMIT — error-handling + deployment config, live staging backend URL

Biswanath

FRONTEND

- Responsive and mobile polish pass — assume judges will look at this on a phone
- Deploy the frontend

COMMIT — responsive UI fixes, live deployment URL

Adhiraj

PRESENTATION

- Submit the idea write-up and PPT on the SIH portal before the 20 September deadline
- Prepare a 2–3 minute demo video
- Get feedback from a faculty mentor or SPOC if one is available

COMMIT — `docs/sih-idea-submission-final.pdf`, demo video link

Tiyasha

DATA & INTEGRATION

- Expand real data coverage to more coastal sectors and zones
- Stress-test the data functions against edge cases — missing data, API downtime
- Finalise the evaluation query set with expected answers

COMMIT — expanded data coverage, `docs/eval-results.md`

Samprikta

AGENT DEVELOPMENT

- Run the full evaluation query set through the agents and tune prompts based on failures
- Add explainability detail to the synthesizer's output — which specific data justified which specific claim
- Write basic automated tests for the agent pipeline

COMMIT — `tests/test_agents.py`, prompt refinements, explainability output

DEADLINE

Idea submitted on the SIH portal by 20 September. Everything else this week works around that fixed date.

Week 5

24 – 30 SEP · PHASES 06 & 07

Goal: a stable, rehearsed, fully working demo and complete documentation — ready to show anyone, at any time, without last-minute surprises.

Dhrubojoyti

LEAD · ARCHITECTURE

- Final architecture review and resolution of any remaining integration issues
- Write the final README with complete setup instructions

COMMIT — final README.md, v1.0 git tag

Isheeka

BACKEND · SUPABASE

- Final backend stability pass and logging clean-up
- Ensure Supabase data is clean and properly seeded for the live demo

COMMIT — stability fixes, demo-ready seed data script

Biswanath

FRONTEND

- Final UI polish based on feedback from rehearsals
- Fix any visual bugs surfaced during rehearsal

COMMIT — final UI polish commits

Adhiraj

PRESENTATION

- Run at least two full team rehearsals of the live demo, timed
- Prepare answers to likely judge questions
- Finalise the pitch deck

COMMIT — docs/pitch-deck-final, docs/anticipated-questions.md

Tiyasha

DATA & INTEGRATION

- Final data QA — confirm demo queries return real, sensible, current data
- Prepare a cached fallback dataset in case live APIs are unreachable during the actual demo

COMMIT — data/demo_fallback_cache.json, final data QA notes

Samprikta

AGENT DEVELOPMENT

- Final test pass on all sample and edge-case queries
- Freeze prompt changes ahead of the demo to avoid last-minute regressions
- Document known limitations honestly, for the team's own Q&A prep

COMMIT — final test pass, docs/known-limitations.md

END STATE

Repo tagged v1.0. Demo rehearsed twice. Every team member can explain any part of the system if asked — that's the actual bar, not just "it works."

SOURCE & COMPILATION NOTE

The problem statement text on page 2 is reproduced from the official Smart India Hackathon 2026 listing (SIH26176, sih.gov.in), current as of the compilation date on the cover. Everything else in this dossier — the planning assumptions, roadmap, diagrams, and five-week task delegation — is original planning material prepared for Team ASTID and is not part of the official listing. Data source names (MOSDAC, INCOIS) reflect what was publicly documented at compilation time; confirm current API access and terms of use directly before the team depends on them.